

AI Campaign Operations Platform

Salesforce Media Cloud build for AppExchange packaging. This document explains, in full, everything designed, built, tested, and left open across the whole project — read it top to bottom before touching the org or the repo.

Org alias MYORG

Source root force-app/main/default

Owner Mohammed Raqib

Status as of 2026-09-28

CONTENTS

- | | |
|-------------------------------------|---------------------------------------|
| 01 The vision & the end-to-end flow | 09 Availability check & delivery sync |
| 02 The one architecture rule | 10 Multi-tenant onboarding |
| 03 Intake → Opportunity | 11 Data model & "no new objects" rule |
| 04 Media planning & approval | 12 Platform gotchas worth knowing |
| 05 GAM connector (this org, live) | 13 Status board — what's done vs open |
| 06 Order push / delete to GAM | 14 File map |
| 07 Optimization Agent | 15 Where to start |
| 08 Amendment Quote flow | |

01 The vision & the end-to-end flow

An enterprise Campaign Planning & Media Operations platform built natively on Salesforce Media Cloud, intended for the Salesforce AppExchange. Publishers/SSPs (Nexstar, CW, Versant, Inmobi, etc.) install it and resell it to their own advertiser clients.

Advertiser request (Form / Email) → **Case** → validation → **Account / Contact / Opportunity** → **Product recommendation** → **Media Plan** (Quote + AdQuote, human-approved) → **Order** → **Push to Google Ad Manager** →

ongoing **Optimization Agent** monitoring → human-approved **Amendment Quote** → applied back to the Order → re-pushed to GAM → delivery stats synced back into Salesforce.

Non-negotiable constraint: zero new custom Salesforce objects. Everything is standard objects (Account, Contact, Opportunity, Quote, Order, Case, Product2, Pricebook2, plus native Media Cloud objects like AdQuote/AdQuoteLine/AdOrderItem) with custom fields, permission sets, and Apex layered on top. Held to strictly across the whole build.

Who actually uses this: real users never type a Line Item ID. They ask "how is Nike performing" or "is the Nike order on track." The agent resolves by name with disambiguation, never by ID, as the primary interface.

IMPORTANT, EASY TO MISS

The Optimization/Forecast agents currently reason over **CSV-simulated** advertiser data (e.g. "Advertiser 007", "9100063352") loaded via Salesforce Files, completely separate from the **real** Salesforce Orders built through the actual intake pipeline (Nike, Acme, Global Media, etc.). There is zero advertiser overlap between the two datasets today. Asking the Optimization Agent about "Nike" returns no match — that's correct behavior given the current architecture, not a bug. Replacing the CSV layer with real GAM-backed data is the natural next big step (see §13).

02 The one architecture rule

Apex is a deterministic evidence engine. The agent (Atlas) interprets and explains. A human approves anything consequential.

Apex never invents a value, never silently picks a "final recommendation," and never writes a real, live change without an explicit human confirmation click. Every write action in this platform — applying a suggestion, creating an Order, pushing to GAM, applying an approved amendment — is gated behind `isConfirmationRequired` / `require_user_confirmation` in the agent, producing a real native Confirm/Cancel card. This rule was followed without exception across the entire build; every time it was tempting to shortcut it (e.g. having Apex auto-classify inventory eligibility), the fix was to give the agent richer evidence, not to let Apex start making the actual decision.

03 Intake → Opportunity

Two live channels feed the same deterministic validation pipeline.

Case-based intake LIVE

`CampaignCaseTrigger` / `CampaignCaseTriggerHandler` fire after insert/update on Case. When a Case moves to Validated, `CampaignIntakeService.validate()` checks required fields, then

`CampaignIdentityResolutionService.resolve()` finds-or-creates the Account/Contact/Opportunity chain. Verified live: a single Case insert auto-created a real Opportunity.

Email-to-Case intake **APEX DONE, SETUP STEP PENDING**

`CampaignEmailIntakeHandler` (`Messaging.InboundEmailHandler`) creates a real Case (Campaign_Request record type) from an inbound email, populating only what an email genuinely gives for free. It deliberately does *not* try to guess structured fields (budget, dates) out of free text — it lets the same validation pipeline flag what's missing.

MANUAL STEP STILL NEEDED

The actual Email Service registration (Setup → Email Services → New Email Service, pointing at `CampaignEmailIntakeHandler`) could not be deployed via Metadata API — `EmailServicesFunction` 's XML has strict undocumented element ordering. This is a genuine 2-minute manual Setup click the org owner still needs to do. The Apex logic is deployed and proven correct independent of this.

04 Media planning & approval

The `Campaign_Planner` subagent, conversational, three-step, each write human-confirmed.

STEP	ACTION	WHAT IT DOES
1	<code>Recommend_Products_for_Opportunity</code>	Read-only ranked product suggestions from real Pricebook data.
2	<code>Create_Media_Plan_from_Selected_Products</code>	Human-confirmed. Creates a Draft Quote + AdQuote/AdQuoteLine.
3	<code>Convert_Approved_Media_Plan_to_Order</code>	Human-confirmed. Only runs once the Quote is Approved/Accepted. Creates a real Order + OrderItems + AdOrderItems.

Real Approval Process (replaces manual status editing) **LIVE**

`Quote.Media_Plan_Approval` — a single-step Salesforce Approval Process on the standard Quote object. Entry criteria: `Status = 'Needs Review'` . Two field-update actions flip Status to `Approved` or `Rejected` . "Submit for Approval" was added to the Quote page's Highlights Panel. Verified live end to end through the real UI: Draft → Needs Review → Submit → routed to approver → Approved → Status auto-flips.

REAL PRICING BUG, FIXED

Every recommended product showed \$0.00. Root cause: `CampaignProductSelectorService` correctly reads the **Standard** Price Book, but that pricebook's entries genuinely had `UnitPrice=0` — real prices only existed on

separate custom rate-card pricebooks. Fixed with a one-time data migration copying each product's real price onto its Standard entry (31 of 70 entries updated). Verified live with real prices (\$8, \$10, \$12...) after the fix.

05 GAM connector — this org, verified live

Real OAuth JWT-Bearer authentication against Google Ad Manager, confirmed with a genuine live API call, not simulated.

```
NETWORK_DISPLAY_NAME: DataBeat Consulting Private Limited
NETWORK_CODE: 23201882692
CURRENCY: USD
TIME_ZONE: America/New_York
```

What's configured in the org

- **Certificate** `gam_service_account_key` — self-signed, imported via Setup → Certificate and Key Management → Import from Keystore.
- **External Credential** `GAM_API_Cred_OAuth` — Protocol `OAuth 2.0`, Flow Type `JWT Bearer Flow`. A manually-added custom JWT claim `scope = https://www.googleapis.com/auth/dfp` was required — Salesforce does not auto-embed it.
- **Named Credential** `GAM_API` → `https://ads.google.com/apis/ads/publisher/v202608`.
- **Apex stubs** — six WSDL2Apex-generated SOAP clients: Network, Company, Order, LineItem, User, Inventory, Forecast services.

THE TWO PITFALLS THAT COST THE MOST TIME

1. Salesforce's plain `JWT` Authentication Protocol is *not* Google's OAuth JWT Bearer flow — it just sends your own signed assertion as the Bearer token, unexchanged. You need Protocol = `OAuth 2.0` + Flow Type = `JWT Bearer Flow` specifically for Salesforce to perform the real token exchange.
2. Even with the right protocol, Google rejected the exchange with `invalid_scope` until a `scope` claim was manually added as a custom JWT Body Claim on the External Credential — the Scope field on the form does not do this automatically for Google's flow.

WSDL2APEX GOTCHA (EVERY GOOGLE-FAMILY WSDL)

Any generated class literally named `*Exception` fails to compile ("Exception class must extend another Exception class") — Apex special-cases the suffix. Fix: rename them (`ApiExceptionInfo`, etc.) after confirming nothing else references the original name.

An older, broken External Credential called `GAM API Cred` (plain `JWT` protocol) still exists — do not use or resurrect it.

06 Order push / delete to Google Ad Manager

Two ways in: conversationally through the agent, or a button directly on the Order record.

`GAMOrderPushService.cls`

Finds/creates GAM Company + Order, builds real Line Items from `OrderItems`, writes GAM IDs back to `AdOrderItem`.

`GAMOrderDeleteService.cls`

Deletes the GAM Order; clears the Salesforce-side GAM fields on success.

`GAMOrderButtonController.cls`

@AuraEnabled wrapper for the Lightning button path, with real status detection.

`gamOrderAction`

(LWC) Push/Delete button on the Order record page with a confirmation modal.

Verified live: pushed a real Salesforce Order → real GAM Order `4204715699` with 2 real Line Items; deleted it and confirmed via a real `getOrdersByStatement` query it returns 0 results.

Real Ad Unit / creative-size mapping

Three fields on `Product2`: `GAM_Ad_Unit_Name__c`, `GAM_Creative_Width__c`, `GAM_Creative_Height__c`. When set, the push resolves the real GAM Ad Unit by name and uses the real creative size; falls back gracefully to "first ACTIVE Ad Unit" / 300x250 when unset or not found in GAM — never throws for a bad name. Verified live: a configured product produced a real Line Item with `CREATIVE_SIZE: 728x90` and the exact targeted Ad Unit ID, not the fallback.

Honest limitation: there's no automatic targeting-mapping engine — this is a per-product config field an admin fills in. This GAM network's real Ad Units are generic news-site placements with no naming relationship to this org's actual product catalog, so there was no natural key to auto-derive one from.

ABSTRACT XSD TYPE GOTCHA

GAM's `performOrderAction` (used for delete) needs an explicit `xsi:type` on the wire for its abstract `orderAction` element. WSDL2Apex's generated stub cannot serialize this correctly no matter how the wrapper field is declared. Fixed by hand-building a raw SOAP envelope and POSTing it directly (still through the same Named Credential, so auth is unaffected). Any future action on this same abstract element needs the same raw-SOAP treatment.

@AURAENABLED VS @INVOCABLEVARIABLE

Lightning silently strips every field from an `@AuraEnabled` method's return value unless each field is *also* marked `@AuraEnabled` — marking a field `@InvocableVariable` only (for Flow/Agentforce) is not enough. This caused confusing "Push Failed" toasts on calls that had actually succeeded. Fixed by mapping into separate local DTOs inside the LWC controller.

07 Optimization Agent

Continuously analyzes CSV-simulated delivery/inventory/targeting/contention data and surfaces suggestions for human review. Apex computes evidence; the agent frames it; nothing changes without a human clicking Confirm.

Response format **REWRITTEN THIS SESSION, DEPLOYED**

Replaced a long multi-section report with a fixed, crisp 4-part format:

1. One line: Line Item, Advertiser, Health, Urgency.
2. One or two sentences on the single biggest driver — not every metric.
3. A numbered list of 1-3 concrete suggestions (numbering matters — "apply suggestion N" depends on it).
4. A closing line: "Analysis only — no changes made. Human approval required to act on any suggestion."

Deep-dive detail (targeting, benchmarks, contention) is now answered only if the user actually asks for it, instead of front-loaded into every response.

Evidence engine — real inventory-eligibility scoring

`CampaignOptimizationService.determineInventoryEligibility()` compares an inventory row's placement/DMA/device/creative-size against active targeting rows and returns a real tiered result (multi-dimension match / single-dimension match / not proven), replacing a previous hardcoded "not proven" for every single row. This gives the agent genuinely defensible alternative-inventory candidates instead of always saying "cannot prove eligibility."

TWO REAL BUGS FOUND AND FIXED WHILE BUILDING THIS

1. Advertiser/line-item name matching was one-directional ("Advertiser 007 lifetime campaign" failed to resolve while bare "Advertiser 007" worked) — fixed to match both directions.
2. The resolver never returned real Line Item IDs behind a multi-match advertiser — only a count, causing the agent to invent placeholder or hallucinated IDs. Fixed by adding a real `candidateLineItemIds` output field.

08 Amendment Quote flow — this session's main build

The full loop your lead asked for: a suggestion gets approved → a real amended proposal is generated → the client approves it → the real Order is updated → GAM is updated.

Suggestion confirmed → **Amendment Quote** created (carries the Order's current lines + the new suggestion) → client reviews & approves via the real Approval Process → **Order synced** to match the approved Quote exactly → queued **push to GAM** reflects the change in the real ad server.

1 • Creating the Amendment Quote — `CampaignSuggestionApplicationService.apply()`

When a human confirms a suggestion, the service checks whether a real Order already exists for that advertiser:

- **Order exists** → creates an **Amendment Quote**: copies the Order's current OrderItems as QuoteLineItems, adds the new suggested line, sets `Status = 'Needs Review'`, tags `Quote.Amends_Order__c` (new lookup field) pointing at the Order it amends.
- **No Order yet** → falls back to a standalone Draft Quote for just the suggestion (original behavior).

OPPORTUNITY-LINKING BUG, FIXED

Amendment Quotes were initially landing on a throwaway "Advertiser - Optimization Suggestions" Opportunity instead of the Order's real original one. Fixed by resolving the Opportunity via `Order.QuoteId` → `Quote.OpportunityId` whenever an existing Order is found. Verified live: a new Nike Amendment Quote correctly landed on the real "Org setup opp" Opportunity.

2 • Applying the approved amendment

— `CampaignAmendmentApplicationService.applyApprovedAmendment()`

Once a human/client approves the Amendment Quote through the real Approval Process:

1. Verifies the Quote is actually Approved/Accepted and actually an amendment (`Amends_Order__c` populated) — refuses otherwise with a clear message.
2. **Syncs the real Order's OrderItems to exactly match the approved Quote**, matched by product: shared products get Quantity/UnitPrice updated, new Quote-only products get inserted, Order-only products dropped from the amendment get deleted. This is the actual mechanism by which an underperforming line gets overruled.
3. Enqueues a Queueable job that pushes the now-updated Order to GAM via the existing push service (kept out of the same transaction as the sync DML — Salesforce forbids a callout immediately after DML in one transaction).

`GAMOrderPushService.createGamLineItems` was upgraded alongside this: previously, a same-named existing GAM Line Item was left completely untouched on re-push. It now calls `LineItemService.updateLineItems` so a re-push genuinely carries the new goal/cost/end-date through to the real ad server.

VERIFIED LIVE, FULL LOOP, REAL DATA

Created a fresh Amendment Quote for Nike → submitted through the real Approval Process → approved → applied → Order 00000108's OrderItems synced correctly → queued push succeeded against real GAM (Order 4206270443, 2 real Line Items).

3 • Root-router misrouting — made safe, not fully fixed

KNOWN, REAL, OPEN LIMITATION

Live testing showed Agentforce's root router (`sfdc_ai__DefaultEinsteinHyperClassifier`) consistently sends "amendment...approved...apply to order" phrasing to `Campaign_Planner` 's Order-conversion action instead of the correct amendment action — even after adding explicit routing rules naming "amendment" phrasing. This once created a real duplicate Order before being caught and deleted.

Real fix applied: a hard guard was added directly in `CampaignOrderConversionService.cls` — it now refuses to convert any Quote with `Amends_Order__c` populated, with a message redirecting to the correct action. Re-tested live: the agent still initially misroutes, but Apex now safely refuses and no bad data is created. **The misrouting itself is unresolved** — worth revisiting if the wrong-action clicks keep happening.

4 • Links in chat — investigated and shelved

CONFIRMED PLATFORM LIMITATION, NOT A BUG IN OUR CODE

Real clickable Quote/Order links were attempted (two approaches: plain URLs in text, then a rich-text HTML output field rendered as its own card). Both were blocked: Agentforce redacts any URL-shaped string to the literal text `URL_Redacted` — confirmed this happens even inside a raw HTML `href` attribute passed through a registered action output. No configurable Setup toggle was found (checked Einstein Setup, Agentforce Gateway Policies — neither is the right control). **Decision made by the user:** skip clickable links for now; the agent instead always states the plain Quote ID and Order ID verbatim in chat, which is confirmed to survive reliably.

09 Availability check & two-way delivery sync

Pre-sale availability check BUILT, BLOCKED BY NETWORK DATA

`GMAvailabilityCheckService.checkAvailability(quoteId)` — for every `AdQuoteLine` on a media plan, builds a prospective (never-saved) GAM Line Item and calls

`ForecastService.getAvailabilityForecast` , returning a deterministic `FULLY_AVAILABLE` / `PARTIALLY_AVAILABLE` / `UNAVAILABLE` classification. Apex computes the comparison; it does not recommend anything.

GENUINELY BLOCKED, NOT A CODE DEFECT

Every call — across multiple real media plans and Ad Units — returns GAM's real

`ForecastingError.NO_FORECAST_YET` . Proven network-wide, not query-specific: even a bare-minimum test against the network's first Ad Unit hits the same error. GAM's own forecasting job hasn't accumulated enough historical traffic on this test network yet. This cannot be fixed from the Salesforce side — it needs either time on this network or a different network with an established forecast history to verify against.

Two-way delivery sync LIVE

`GAMDeliverySyncBatch` (Batchable, scope 25) pulls real delivered impressions/clicks/status from GAM's `LineItemService` for every `AdOrderItem` genuinely pushed to GAM, batching multiple Line Item IDs into one query per chunk. `GAMDeliverySyncSchedulerJob` runs it hourly or on demand. Kept deliberately separate from the CSV-backed Optimization pipeline so that pipeline is never disturbed.

Verified live against 12 real `AdOrderItem` records — 0 errors, real `GAM_Delivery_Status__c` and `GAM_Last_Synced__c` values confirmed by direct query.

BATCH APEX CALLOUT LIMIT

Batch classes only get 1 callout per `execute()` unless they also implement `Database.AllowsCallouts` — the first real callout failed with `Too many callouts: 1` , an easy one to miss since the code compiles and even runs its first chunk before failing.

10 Multi-tenant onboarding automation

Everything in §05 was configured manually by a technical person. A real AppExchange client's admin cannot run `openssl` / `keytool` commands. This solves that, entirely in pure Apex — no external service, no Python, no hosting, per explicit requirement from the user's manager/VP.

SECURITY MODEL, DELIBERATELY CHOSEN AND RESEARCH-BACKED

Per-client Google service account (not a shared "log in with Google" OAuth consent flow) — published guidance on real GAM add-ons treats service-account auth as generally safe and well-aligned with how enterprise IT actually audits/revokes API access via their own GCP IAM console.

Phase 1 — `GAMJKSBuilder.cls`

DEPLOYED, VALIDATED AGAINST REAL KEYTOOL

A pure-Apex Java KeyStore (JKS) binary writer — Salesforce's Setup UI only accepts genuine JKS, not PKCS12, and no API exists to upload a keystore programmatically, so it has to be generated and handed to the client pre-built. Given a client's pasted service-account JSON, it extracts the private key, builds a self-signed X.509 certificate from scratch, encrypts the key with Java's proprietary JKS algorithm, and assembles the full binary container including its integrity hash — all done on hex strings since Apex Blobs aren't byte-indexable.

Validated by round-tripping a generated JKS through real OpenJDK `keytool` : parsed correctly, and the recovered private key's RSA modulus was byte-identical to the original.

Phase 2 — `GAMOnboardingProvisioningService.cls`

DEPLOYED, IDEMPOTENT REDEPLOY VERIFIED LIVE

After the one manual keystore import, this auto-provisions the rest via a self-targeted Metadata API deploy (using the running admin's own session ID — no Remote Site Setting needed) of the `ExternalCredential/NamedCredential/PermissionSet`, using `ApexMinimalZip.cls` (a from-scratch, uncompressed-entry ZIP writer with its own CRC-32 implementation) to build the deploy payload, since Apex has no compression API. It then auto-detects the client's GAM network code via `getAllNetworks()` (the one GAM operation that works before a network code is known) and persists it to a new `GAM_Config__c` hierarchy custom setting.

The `gamOnboardingWizard` LWC drives this end to end with a real status spinner, ending on a "connected" screen showing the detected network.

PERMANENTLY MANUAL — CONFIRMED VIA DIRECT API TESTING

The literal "Import from Keystore" click in Setup has no API path at all (verified: the Certificate Tooling API object has zero fields for key material). A client's own IT admin adding your service account inside their own GAM Admin panel, and generating that service account JSON in Google Cloud Console in the first place, also have to happen on the client/Google side. No workaround exists on the Salesforce platform for any of these three.

STILL OPEN

Only the idempotent "redploy over already-working config" path has been verified live. The true first-time-create path, in an org that starts with none of this metadata, has not yet been tested end to end.

11 Data model & the "zero new custom objects" rule

`Optimization_Change_Log__c` and `Optimization_Change_Request__c` (the original custom objects used for the audit trail) were **deleted** and replaced with the standard **Case** object plus six new custom fields, per an explicit decision to resolve the architecture conflict rather than keep a hybrid.

- A Case with `Optimization_Event_Type__c` populated = a log entry (write-only audit record).

- A Case with `Optimization_Action_Type__c` populated = a change request (scheduler dedupes on `Status = 'Pending Approval'`).
- Case already has a protected "Campaign Request" record type wired to the intake trigger — these new audit Cases use the default/Master record type, and the trigger handler explicitly no-ops for any other record type, confirmed safe live.

TASK DOES NOT ACCEPT CUSTOM FIELDS IN THIS ORG

Custom fields could not be added to the standard Task object at all — rejected by both Metadata API deploy and the Setup UI (no "New" button on Task's field list). Almost certainly a Media Cloud/managed-package restriction on the Activity model. Case was used instead, and works.

Historical audit data (59 log rows + 4 change-request rows) was **not** migrated — a genuine, unexplained org-side issue briefly made the old objects' field values unreadable via every API path right when migration was attempted. The user explicitly chose to delete the objects anyway rather than block on low-value historical data.

12 Platform gotchas worth knowing before you touch anything

AREA	GOTCHA
DML + callout	A callout immediately after DML in the same transaction throws "uncommitted work pending" — wrap the callout in a Queueable enqueued after the DML.
Batch Apex	Needs <code>Database.AllowsCallouts</code> to make more than 1 callout across <code>execute()</code> calls.
Agentforce actions	Deploying the Apex class alone never makes it callable — it must also be registered as a real Agent Action via Agentforce Asset Library. This org's UI has no "Add Output" for an existing action; the reliable fix is delete-and-recreate.
Agentforce outputs	URL-shaped strings anywhere in any output field get redacted to literal <code>URL_Redacted</code> text — no known Setup toggle exists for this.
Integer outputs	A known Agent Action Wizard bug mishandles Integer-typed outputs — <code>lineItemsPushed</code> is deliberately typed Decimal, don't change it back.
FLS disguised as schema error	A brand-new field can say "No such column" in Apex/SOQL purely because of missing Field-Level Security on the running user's permission set — looks identical to a real schema-propagation delay. Check FLS first.
<code>sf apex run</code> logging	Ignores <code>TraceFlags</code> — always logs at a fixed low level. For deep callout debugging, use raw <code>HttpRequest/response</code> printing instead.
Certificate import	Only possible through the interactive Setup UI — no Metadata API, no Tooling API path exists at all.

13 Status board

CAPABILITY	STATUS	NOTES
Case & Email intake	● MOSTLY LIVE	Email Service Setup registration still a manual click.
Product recommendation & pricing	● LIVE	\$0.00 pricing bug fixed and verified.
Media Plan → Order + real Approval Process	● LIVE	Verified end to end through the real UI.
GAM auth (this org)	● LIVE	Real network data confirmed via live SOAP call.
Order push / delete to GAM	● LIVE	Idempotent, safe re-push, real Ad Unit mapping.
Optimization Agent — crisp responses	● DEPLOYED	Not yet re-tested live in a fresh conversation this session.
Amendment Quote creation	● LIVE	Correct Opportunity linkage confirmed.
Apply approved amendment → sync Order → push GAM	● LIVE	Full loop verified with real records.
Root-router "amendment" misrouting	● OPEN, MADE SAFE	Apex guard prevents bad data; routing itself still wrong sometimes.
Clickable links in chat	● SHELVED	Confirmed platform limitation; plain IDs used instead.
Pre-sale availability check	● BLOCKED EXTERNALLY	Code correct; this GAM network has no forecast data yet.
Two-way delivery sync	● LIVE	Hourly scheduler verified against real GAM stats.
Multi-tenant JKS builder	● VALIDATED	Round-tripped against real keytool successfully.
Multi-tenant auto-provisioning	● IDEMPOTENT PATH ONLY	True first-time-create path not yet tested.
Optimization Agent on real (not CSV) data	● NOT STARTED	Natural next step now that GAM auth works.

14 File map

Everything lives under `force-app/main/default/`. The most important Apex classes, by area:

`CampaignIntakeService.cls`

Validation on new Cases.

`CampaignIdentityResolutionService.cls`

Account/Contact/Opportunity resolution.

`CampaignEmailIntakeHandler.cls`

Inbound email → Case.

`CampaignProductSelectorService.cls`

Product recommendation.

`CampaignMediaPlanService.cls`

Draft Quote/AdQuote creation.

`CampaignOrderConversionService.cls`

Approved Quote → Order (+ amendment guard).

`GAMNetworkServiceClient.cls`

Auth/network check — the standing regression test target.

`GAMOrderPushService.cls`

Push Order to GAM.

`GAMOrderDeleteService.cls`

Delete GAM Order.

`GAMOrderButtonController.cls`

Record-page push/delete button.

`GAMAvailabilityCheckService.cls`

Pre-sale forecast check.

`GAMDeliverySyncBatch.cls`

Real delivery stats sync-back.

`CampaignDataProvider.cls`

CSV-backed simulated delivery data.

`CampaignForecastService.cls`

Forecast evidence engine.

`CampaignOptimizationService.cls`

Optimization evidence engine.

`CampaignSuggestionApplicationService.cls`

Apply suggestion → Amendment Quote.

`CampaignAmendmentApplicationService.cls`

Apply approved amendment → sync Order → push GAM.

`GAMJKSBuilder.cls`

Pure-Apex JKS keystore writer.

`ApexMinimalZip.cls`

Pure-Apex ZIP writer for Metadata API deploys.

`GAMOnboardingProvisioningService.cls`

Auto-provision credentials for a new client.

`agentscript.txt`

Source of truth for the agent's reasoning instructions.

`Campaign_Forecasting_Agent.agent`

The deployed mirror of agentscript.txt.

`test_gam_still_works.apex`

Standing regression check — run after any change.

15 Where to start if you're picking this up

1. Read `PROJECT_HANDOFF.md` at the repo root — this artifact is a guided summary of it; the markdown file has the full narrative detail and every real record ID referenced along the way.
2. Run `test_gam_still_works.apex` immediately to confirm GAM auth and the Optimization Agent's core plumbing are intact before changing anything.
3. Pick a thread from the status board (§13) marked ● OPEN or ● PARTIAL — the root-router misrouting fix and running the Optimization Agent against real (not CSV) GAM data are the two highest-value next steps.
4. Whatever you change, keep `agentscript.txt` and the deployed `.agent` bundle in sync, validate with `sf agent validate authoring-bundle`, and re-run the standing GAM regression check afterward. This was the working discipline for the whole project — don't skip it.

Internal engineering handoff — Media Cloud AppExchange platform. Compiled from the full build history in `PROJECT_HANDOFF.md`. Every "verified live" claim in this document was confirmed against real Salesforce/GAM data, not assumed.